Alcifra

Tube — Module Manual

Your own videos on your own installation, played by PeerTube clients

1 What the module does

A **Tube Channel** is a folder that clients show as one channel, and a repository can hold several. A video is a document wearing the **Video** mask. Prepare it anywhere; it is offered only through a channel, by sitting in one or by a reference placed in one, so one video can be in several channels. With its **Published** field at `yes`, PeerTube clients play it from your video host, streamed straight from storage; set it back to `no`, or remove it from the channel, and it is gone from the clients at once. Outside every channel a video is shown nowhere, whatever **Published** says.

Publishing takes two rights: editing the video, and creating in the channel. In the clients' overview a video appears once per channel it is in; opening one names the channel it sits in, else the one that referenced it first.

2 Setting it up

1. Under **Modules** ▸ **Tube**, enter the **Video host** (e.g. `video.example.com`) your proxy sends to the installation's `peertube` prefix, and the instance's name and description clients show.
2. Create a folder with the **Tube Channel** mask. It holds videos and their thumbnails, and references to them. Its **Description** is shown by clients.
3. File a video in the channel or anywhere else, with the **Video** mask: `.mp4`, `.m4v`, `.webm`, `.mov`, `.mkv` or `.ogv`. An MP4 plays everywhere; other types depend on the client, because files play as stored. Fill **Description** and, when you release it, **Published** = `yes` and **Published at** (the catalogue lists newest first). **Duration seconds**, **Language** and **Tags** are shown by clients.
4. A **Thumbnail** points at a document with the **Video Thumbnail** mask (`.jpg`, `.jpeg`, `.png`, `.webp`), wherever it lies.
5. If the video was prepared outside a channel, place a reference to it in the channel. The module may then read that video and its thumbnail, and nothing else in that place.
6. The module also needs read access to the channel itself: grant **Module: Tube** on the channel or on a folder above it.

3 The pages in a browser

The video host's front page shows the latest videos, the newest in the upper left corner and the oldest in the lower right, 24 to a page, each with its thumbnail and description. Each video links to its channel's page, which shows that channel's latest videos the same way. A video in two channels appears once for each. The front page links to a short FAQ on watching in a mobile app.

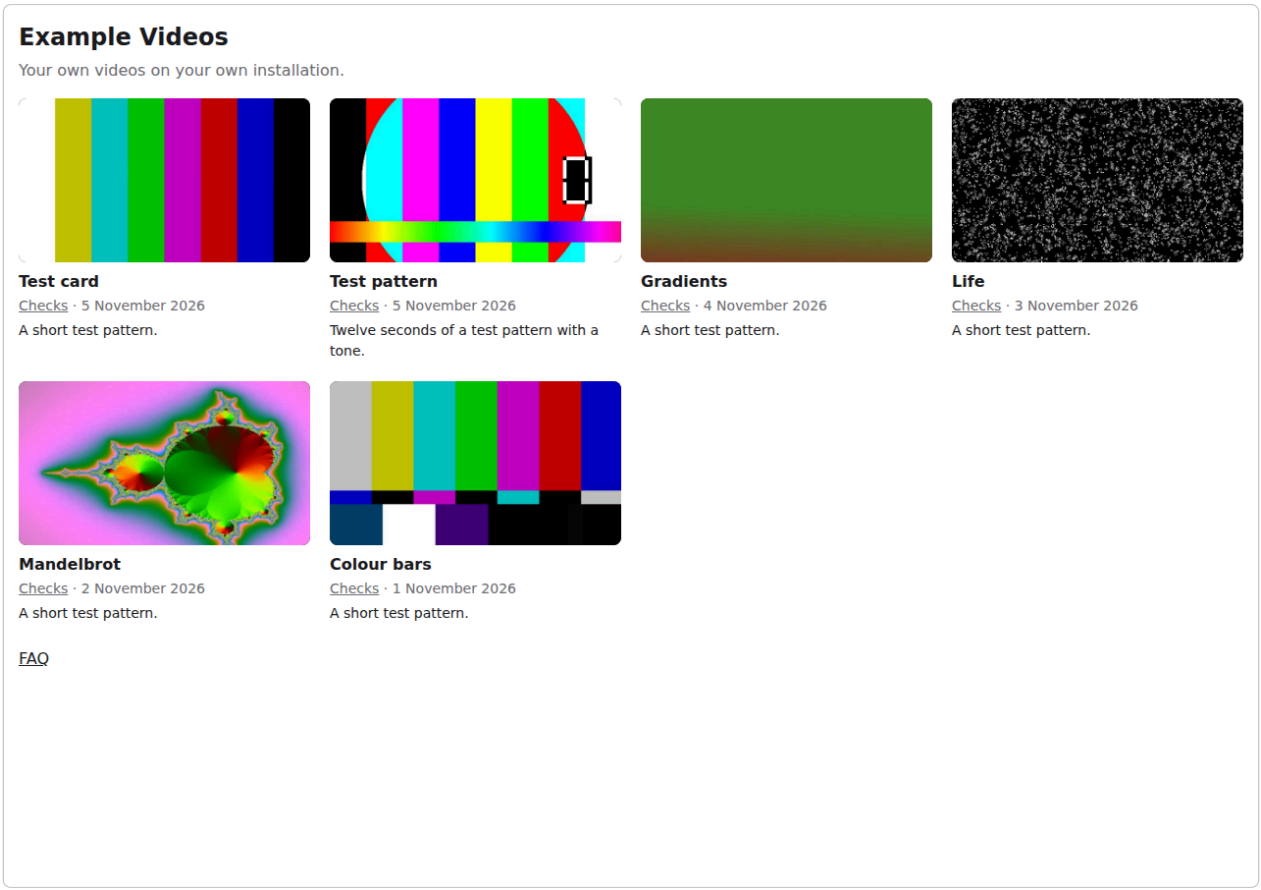


Figure 1: The video host's front page: the latest videos, newest first.

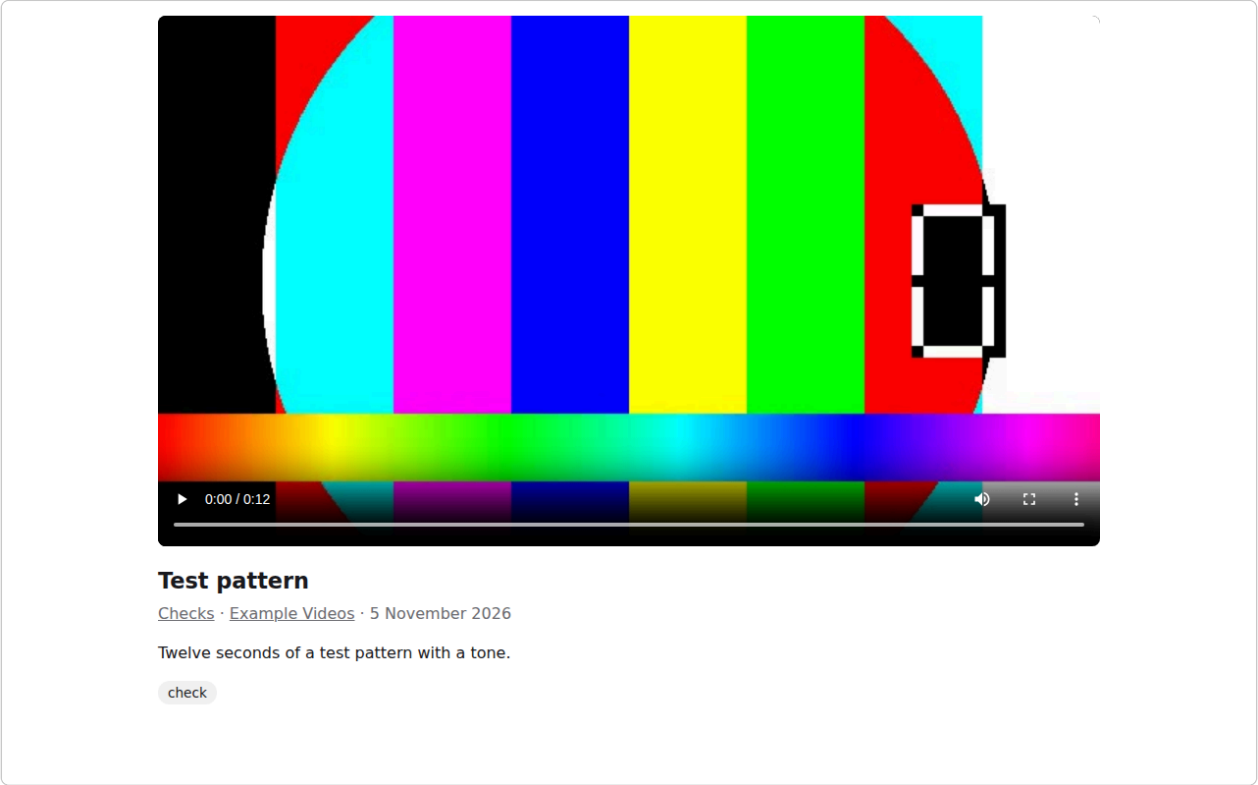


Figure 2: A video's page, around the browser's own player.

The proxy in front of the video host has to pass `/`, `/c/` and `/faq` to the module as well, and each thumbnail counts against the limit on anonymous requests per address.

A **Tube Playlist** is a folder in a channel that holds only references to videos, of any channel you can see: **New ▸ Tube Playlist** on the channel, then place references in it. It is published with its channel, plays in the order the references were placed, and shows only what is published in a channel. The channel's page links its playlists, and PeerTube clients find them under the channel.

4 A video on another website

Any website can show a published video, and its owner needs nothing from you: no account, no key, no signed address. Open the video's page on your video host and copy its address, `https://video.example.com/videos/watch/<id>`. Replace `watch` by `embed`, and the other site places that address in a frame. The embed page holds nothing but the player, and it is the one page of your video host that other sites may frame.

```
<iframe src="https://video.example.com/videos/embed/<id>"
        width="560" height="315" allowfullscreen
        title="The video's title"></iframe>
```

A site that wants its own player points a `<video>` element at the video's file address, `https://video.example.com/files/<id>.mp4` (with the extension the video was filed with), and its poster at `https://video.example.com/files/<id>/thumbnail`. The file address answers every request with a fresh link into your storage. **Never copy that storage link**: it is signed for five minutes only, so a page that kept it would stop playing. The stable addresses above never change while the video stays published.

What the other site should know:

- The video plays only while it is published and sits in a channel. Unpublish it, or take it out of every channel, and every embed stops at its next load.
- Its visitors' browsers fetch the page and the file from your video host, and the video's bytes from your storage directly. So their addresses reach your installation, and each request counts against the limit on anonymous requests per address.
- A site with a Content Security Policy of its own lists your video host under `frame-src` for the frame, or your video host and your storage host under `media-src` for a `<video>` element.
- There is no oEmbed: a site builder that turns a pasted link into a player (WordPress, for one) needs the frame pasted as HTML instead.
- A tenant that encrypts at rest publishes nothing, so there is nothing to embed.

5 The Tube tab

Where the module is active, the clients gain a **Tube** tab or tile. It lists the channels you can see, each with a checkbox and a lock if you can't add to it, and a filter over the videos. Your own rights decide what shows: published videos, and any video you may edit, marked as a draft.

Where your rights allow, you can publish or unpublish a video, edit its details, place it in another channel, and remove it from a channel that only references it, which never deletes the video. Channels are created, renamed and deleted in the Repositories tree. The web and the phone play in the tab; the desktop opens the watch page in the browser, so a draft has nothing to open there. A tenant that encrypts at rest sees the tab empty, with a line saying why.

6 Limits

The module serves tenants that do not encrypt content at rest: a video's bytes go straight from storage to the player, and an encrypted object could only be served through the server. Such a tenant sees an empty catalogue, and the log says why. Files play as stored; there is no transcoding.